CHAPTER 10

Secure Your Application with OpenID Connect

Security is essential in an application, by which I mean that almost all applications need a mechanism to identify the user attempting to perform actions on them. This is called authentication, which should not be confused with authorization, a mechanism allowing privileges to be given to an authenticated user, that is, allowing them to perform specific actions that other users will not achieve. In this chapter, you will learn

- Introduction to OpenID Connect (OIDC)
- Configuring authentication and authorization in ASP.NET Core
- Passing a JSON Web Token (JWT) into requests and getting the user's identity

© Anthony Giretti 2023
A. Giretti, *Coding Clean, Reliable, and Safe REST APIs with ASP.NET Core 8*,
https://doi.org/10.1007/978-1-4842-9979-1_10

Introduction to OpenID Connect

OpenID Connect (OIDC) is an identification standard that is positioned above *OAuth 2.0*, which is itself an authorization protocol. *OpenID Connect* works on the principle of delegating user authentication: with *OpenID Connect*, this responsibility is entrusted to a third-party service. The latter uses the protocol to ensure the user is authenticated, so the application protected by *OpenID Connect* does not know how the authentication is performed. So that's it with the login forms in your code.

To be completely independent of the application, this authentication system can be transverse and reused to develop a single authentication within an information system. This is the very definition of the *Single Sign-On (SSO)* principle. We end up with an interaction between three actors:

1. The client that is a web app, for example
2. The identity provider
3. The protected resource

Figure 10-1 shows the relationship between the three actors.

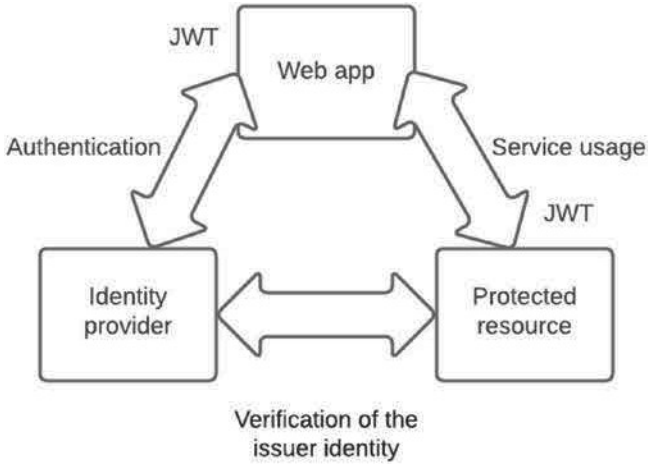

Figure 10-1. *The relationship between the three actors in OpenID Connect*

The client will authenticate with the service provider. The latter will issue a *JSON Web Token (JWT)* that will be used to access the protected resource. This resource will validate the token received by retrieving the metadata from the identity provider to certify that the latter is the issuer of the JWT. Metadata is retrieved only once, and then the application can validate the JWT autonomously. A JWT is a JSON accompanied by a signature and the reference to the key, which allows the signature to be verified. The whole is encoded in Base64, and dots separate the three parts. They are assembled as follows: the header, the content, and the signature. I will show you an example in the next section. **RFC 7519** describes the JWT standard, which can be found at this address: `https://datatracker.ietf.org/doc/html/rfc7519`.

This introduction is brief. The goal is not to teach you *OpenID Connect* in great detail but rather to understand the basic principle, the minimum, to allow you to use *OpenID Connect* to authenticate in ASP.NET Core. If you want to learn more about OpenID Connect, you can consult the official documentation for this protocol here: `https://openid.net/connect/`.

To configure ASP.NET Core with *OpenID Connect*, we must have an identity provider to achieve our ends. You may not know it, but a lot of applications use *OpenID Connect*, and I think you already know of the most often-used identity providers:

- Facebook
- Google
- Apple
- And less frequently, Microsoft

Figure 10-2 shows the canva.com website offering to authenticate with different providers.

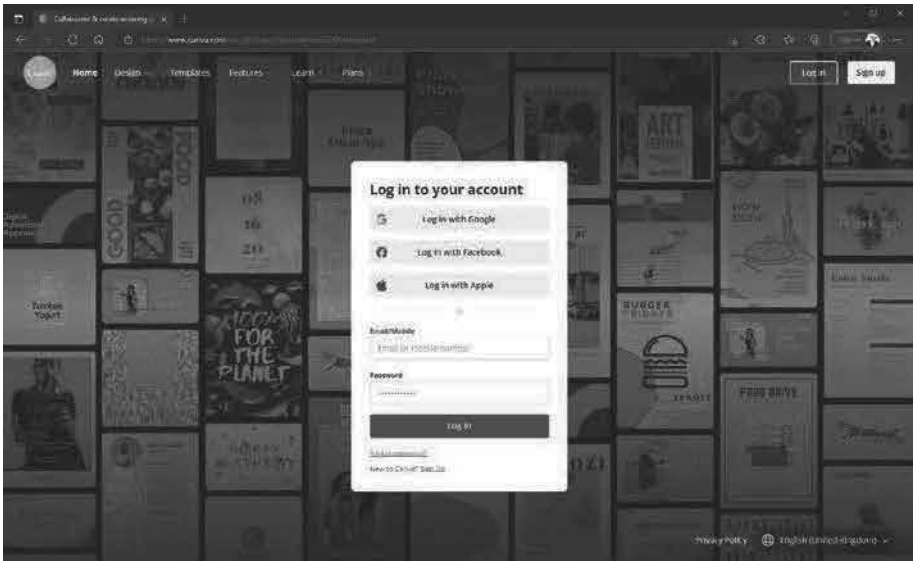

Figure 10-2. *Canva.com uses Google, Facebook, and Apple as OpenID Connect providers*

In the code samples in this chapter, I'll be using the Microsoft authentication platform based on Azure Active Directory. However, I will not go into details about its configuration. I will show you how to configure ASP.NET Core; the authentication part will be up to you. Azure Active Directory and *OpenID Connect* are big pieces. To avoid losing the line on the main subject, I invite you to learn more about the Microsoft Identity Platform here: `https://docs.microsoft.com/en-us/azure/active-directory/develop/active-directory-v2-protocols`. If you want to get things done quickly, you can follow my tutorial on setting up *OpenID Connect* on Microsoft Azure here: `https://anthonygiretti.com/2018/02/28/using-openidconnect-with-azure-ad- angular5-and-webapi-core-introduction/`.

Note Along with this chapter, I will assume you could obtain the emitted **access_token**, which is used as a bearer token.

Configuring Authentication and Authorization in ASP.NET Core

The configuration of an ASP.NET Core API is generic, regardless of the identity provider used to emit a token. For example, the following configuration applies to minimal APIs or other ASP.NET Core applications. To get started, install the required Nuget package: *Microsoft.AspNetCore.Authentication.JwtBearer*.

Once done, go to the *Program.cs* file and configure and activate authentication and authorization.

Configuration needs several lines of code:

- The **AddAuthentication** extension method defines the authentication based on a JWT using the *JwtBearerDefaults.AuthenticationScheme* value. AddAuthentication defines the type of authentication, and it's generic. *JwtBearerDefaults.AuthenticationScheme* is specific to JWT authentication.
- The **AddJwtBearer** extension method allows the setup of the *Authority*, that is, the authentication server address, and *Audience*, that is, the target application for which the JWT is emitted. Both of these values are given by the identity provider you have chosen. Then we will configure the parameters used to validate the JWT: *ValidateLifetime* and *ValidateIssuer* both set to

True and *Clockskew*, which is used to manage the time gap between the JWT issuer and the application and will be set up to 5 min. In other words, the latter allows a 5-minute gap between the JWT expiry timestamp and the application, where the token lifetime is validated.

- The **AddAuthorization** extension method allows configuring authorization in ASP.NET Core by using the *Authorize* attribute.

Activation is only about adding two middlewares in the pipeline:

- The **UseAuthentication** extension method, which registers the *Authentication* middleware in the pipeline
- The **UseAuthorization** extension method, which activates the *Authorization* middleware in the pipeline

Listing 10-1 shows the *Program.cs* properly configured. *Authority* and *Audience* are partially hidden. They are specific to my Azure Active Directory tenant on Microsoft Azure. Following their documentation, you can find the Authority and Audience configuration parameters in *any OpenID Connect* provider. Once again, if you want to try *OpenID Connect* with Microsoft Azure, you can follow the tutorial on my blog post I mentioned earlier in this chapter.

Listing 10-1. Configure and activate OpenID Connect authentication and authorization on ASP.NET Core

```
var builder = WebApplication.CreateBuilder(args);

...

builder.Services.AddAuthentication(options =>
{
```

```
    options.DefaultAuthenticateScheme = JwtBearerDefaults.
    AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.
    AuthenticationScheme;
}).AddJwtBearer(options =>
{
    options.Authority = "https://login.microsoftonline.
    com/136544d9-xxxx-xxxx-xxxx-10accb370679/v2.0";
    options.Audience = "257b6c36-xxxx-xxxx-xxxx-6f2cd81cec43";
    options.TokenValidationParameters.ValidateLifetime = true;
    options.TokenValidationParameters.ValidateIssuer = true;
    options.TokenValidationParameters.ClockSkew = TimeSpan.
FromMinutes(5);
});

builder.Services.AddAuthorization();

...

var app = builder.Build();

...
app.UseCors(); // Before Authentication and Auhtorization
middlewares
app.UseMiddleware<YourMiddleware>(); // Before Authentication
and Authorization middlewares

app.UseAuthentication();
app.UseAuthorization();

...

app.Run();
```

Caution The *UseCors* middleware and other custom middlewares must be placed **before** the *UseAuthentication* and *UseAuthorization* middlewares, or else they won't work correctly since UseCors must not depend on authentication, so authentication must apply after CORS handling.

To apply authentication on your minimal endpoints, you'll need to add the *RequireAuthorization* extension method on any minimal endpoint you want to protect by authentication, like the GET /authenticated endpoint as shown in Listing 10-2.

Listing 10-2. The GET /authenticated endpoint

```
app.MapGet("/authenticated", () =>
{
    return Results.Ok("Authenticated !");
}).RequireAuthorization();
```

This example is the simplest but allows you to do the necessary to get authenticated, but it doesn't show authorization. Very often, applications require higher privileges for some users. Those users may have more responsibilities and may need to perform sensitive actions. The company will assign a particular (or several) role(s) that can be passed in a JWT when the latter is issued and handled by any ASP.NET Core application. Listing 10-3 shows how to configure a custom policy on the *AddAuthorization* method that ensures the authenticated user has the *SurveyCreator* role, and this role will be assigned to the GET /authorized endpoint that requires the *SurveyCreator* custom policy to get executed.

Listing 10-3. The GET /authorized endpoint assigned with the SurveyCreator policy

```
...

builder.Services
        .AddAuthorization(options =>
         options.AddPolicy("SurveyCreator",
         policy =>  policy.RequireRole("SurveyCreator")
));

...

var app = builder.Build();

...

app.UseAuthentication();
app.UseAuthorization();

...

app.MapGet("/authorized", () =>
{
    return Results.Ok("Authorized !");
}).RequireAuthorization("SurveyCreator");

..

app.Run();
```

Suppose a user tries to invoke any endpoint configured with authentication or authorization without a valid JWT. ASP.NET Core will return the HTTP UnAuthenticated (401) error in that case. If the user is authenticated but doesn't meet the authorization requirements, such as missing the proper role, ASP.NET Core will return the HTTP Forbidden (403) error.

I have shown you how to secure your application with a JWT. You must follow your company's business rules to apply the proper criteria to protect your application. Before finishing this section, I would like to show you what a JWT looks like with the *SurveyCreator* role assigned to user Anthony Giretti. First, please generate a token with your provider, and then go to the `https://jwt.io` website to observe the content of your JWT. Figure 10-3 shows the JWT of my decoded provider.

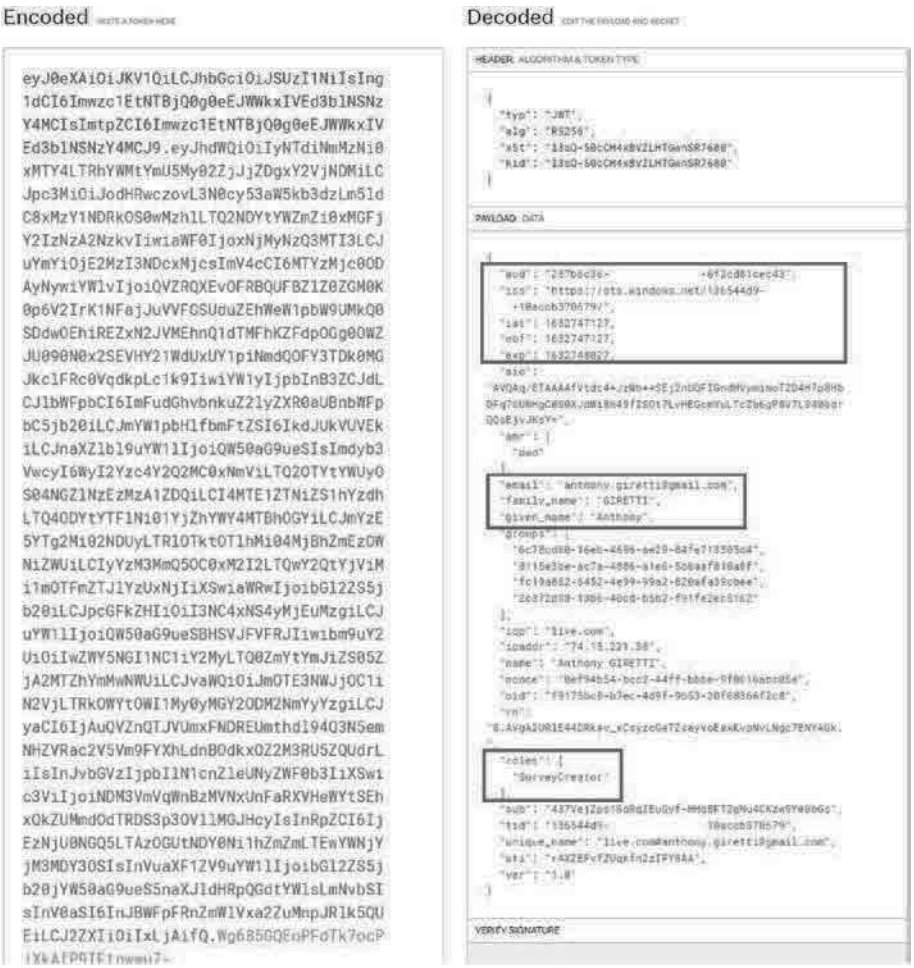

Figure 10-3. *A JWT decoded on the `https://jwt.io` website*

As you can see, we find the information relating to the provider and the expiration date of the JWT in the first framed block and information on the user for whom the JWT was issued in the second framed block, then followed by the role(s) that the user may have. So you will have understood that decoding your JWT will be used to debug your application if you have trouble with the expiration date of your JWT, if you are using the wrong *Audience,* or if you are not using roles correctly (or if you have poorly set up your JWTs with your identity provider).

In the next section, I'll show you how to use the token and pass it into the headers from Postman and Swagger.

Passing a JWT into Requests and Getting the User's Identity

Passing a JWT to any web application always works the same way. It's a matter of passing a header named Authorization in the headers with the value *"bearer {YourJWT}".*

Note In general, steps to accomplish JWT authentication with OIDC are to set up an OIDC provider, have a UI application that fetches a JWT, and then send it to the back end, such as an ASP.NET Core API. The examples in this section show you how to debug back-end applications (ASP.NET Core), assuming you already have generated a JWT. You can also set up Identity Server, an open source *OpenID Connect* provider. You can learn how to set it if you don't want to use Microsoft Azure or any other providers here: `https://scientificprogrammer.net/2022/10/02/securing-your-signalr-applications-with-openid-connect-and-oauth/`.

If you remember, at the start of the book, I told you about Swagger, which lets you generate documentation for your API and expose it to your customers. Since Swagger is well designed, you can configure it to take a JWT and then pass it to the HTTP request. To do this, you need to configure the *AddSwaggerGen* method, as shown in Listing 10-4.

Listing 10-4. The AddSwaggerGen method configured for accepting a JWT

```
builder.Services.AddSwaggerGen(c =>
{
    c.AddSecurityDefinition("Bearer", new
    OpenApiSecurityScheme()
    {
        Name = "Authorization",
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
        Description = "JWT Authorization header required"

    });
    c.AddSecurityRequirement(new OpenApiSecurityRequirement {
        {
            new OpenApiSecurityScheme {
                Reference = new OpenApiReference {
                    Type = ReferenceType.SecurityScheme,
                        Id = "Bearer"
                }
            },
            new string[] {}
        }
    });
});
```

If we now execute the Swagger page, you should see an "Authorize" button at the top right of the page, as shown in Figure 10-4.

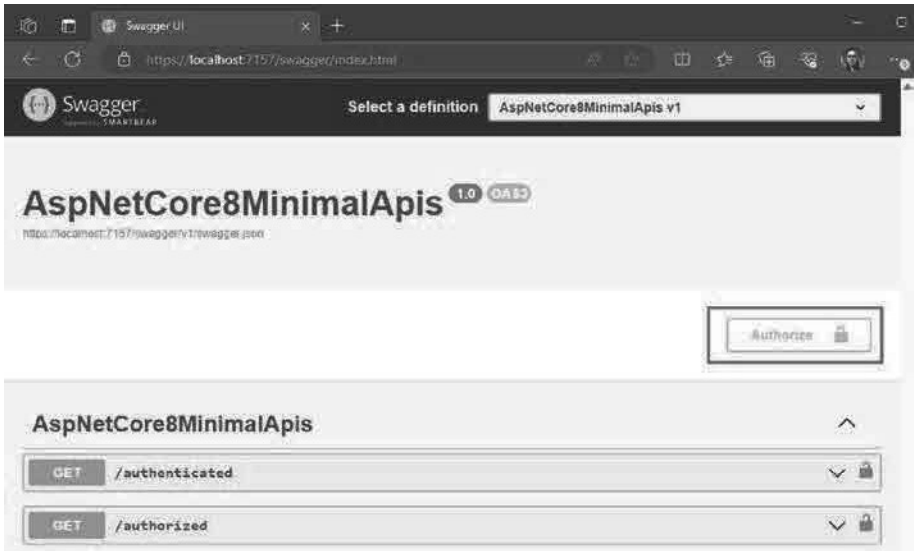

Figure 10-4. *The Authorize button*

If you click, you'll be asked to paste your JWT, taking care not to forget the bearer keyword in front of it, as shown in Figure 10-5.

Figure 10-5. *Filling the headers with the JWT bearer Authorization*

If you run the /GET authorized endpoint, assuming the JWT is valid and contains the *SurveyCreator* role, Swagger should return a successful response as shown in Figure 10-6.

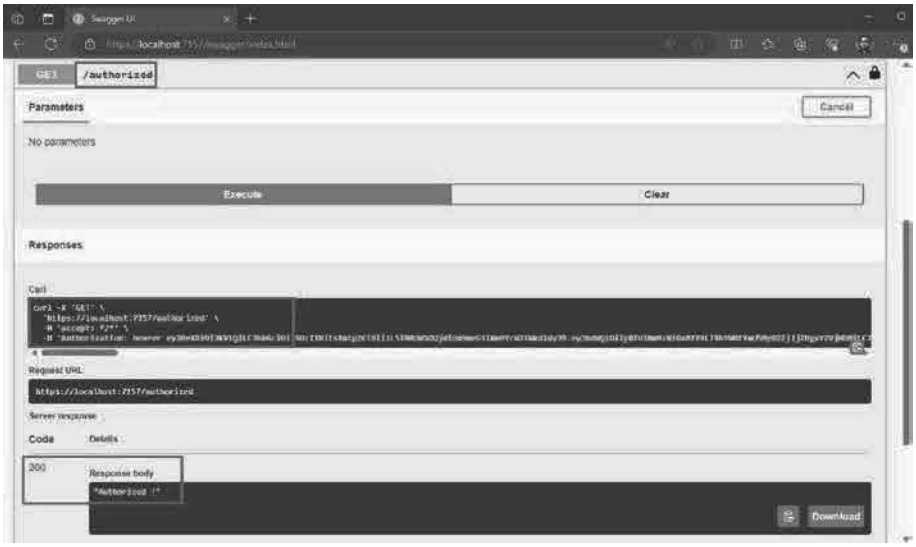

Figure 10-6. *The GET /authorized endpoint output in Swagger after passing a valid JWT*

Now, let's try Postman**. Be careful; Postman itself adds the bearer keyword,** so you can only paste your JWT in the *Auth* section, as shown in Figure 10-7.

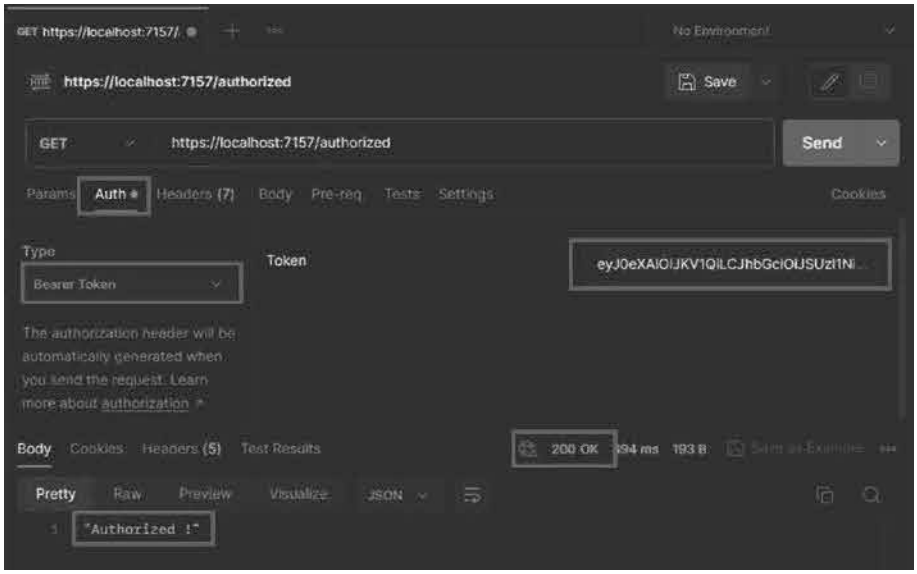

Figure 10-7. *The GET /authorized endpoint output in Postman after passing a valid JWT*

Into your minimal endpoint, inject the *ClaimsPrincipal* class, which is automatically registered in the dependency injection by ASP.NET Core, and you'll get the user's identity in the form of *Claims*, as shown in Figure 10-8.

Figure 10-8. *Accessing the user's identity through the UserClaims object*

When a user is logged in, their identity is defined by *Claims*, which somehow defines their profile as a logged-in user; in other words, they are identity data issued by the identity provider. As you can see from the preceding figure, there is a whole range of claim types, such as the user's name or roles. The claims contain all the information about the user, which is customizable on the identity provider side, and I won't go into detail here. However, we will design a service that will expose user profile data more intelligibly. Let's consider the *UserProfile* class, which implements the *IUserProfile* interface and contains two properties, *Name* and *Roles* (for a simple, easy-to-understand example), as shown in Listing 10-5.

Listing 10-5. The UserProfile class

```
using System.Security.Claims;

namespace AspNetCore8MinimalApis.Identity;

public class UserProfile : IUserProfile
{
    private readonly IHttpContextAccessor _context;
    public UserProfile(IHttpContextAccessor context)
    {
        _context = context;
    }

    public string Name => _context.HttpContext.User?.Claims.
    FirstOrDefault(x => x.Type == "name")?.Value;

    public IEnumerable<string> Roles => _context.HttpContext.
    User?.Claims.Where(x => x.Type == ClaimTypes.Role).Select(x
    => x.Value);
}
```

As you can see, I access Claims using LINQ. It is important to remember that the *ClaimsPrincipal* class is not available directly via dependency injection, as in a minimal endpoint when it's a custom service. Don't forget to register the *IUserProfile* service with its *UserProfile* implementation and the *HttpContextAccessor* service with its *AddHttpContextAccessor* extension method. Let's run the same endpoint as earlier but with the *IUserProfile* interface injected, as shown in Figure 10-9.

Figure 10-9. *Accessing the user's identity through the IUserProfile interface*

Summary

Voila! You know how to manage JWT authentication in your application. Remember that, as I write these lines, JWT authentication is the most widely used way to authenticate to an API, thanks to its reliability and the widespread adoption of *OpenID Connect*. Now you know how to do everything in a moderately complex API. There's just one thing left: test your API. And I'll give you the final chapter of this book.